Experiment 6: Time Series Analysis of Website Traffic

Aim:

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

Dataset:

A dataset containing daily website traffic data with columns such as Date, Page_Views, Unique_Visitors, and Bounce_Rate.

Methodology:

1. Load and Clean Data:
o Import the dataset and parse the Date column as a datetime object.
o Handle missing values by interpolation.
2. Time Series Decomposition:
o Decompose the time series data of Page_Views to identify trend, seasonality, and residuals.
3. Visualization:
o Plot the time series data for Page_Views, Unique_Visitors, and Bounce_Rate.
o Use moving averages to smooth the time series and highlight trends.
o Create seasonal plots to visualize patterns over time.
4. Anomaly Detection:
o Identify and visualize anomalies in the time series data using statistical methods such as the Z-score.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load dataset
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')

# Time Series Decomposition
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
```

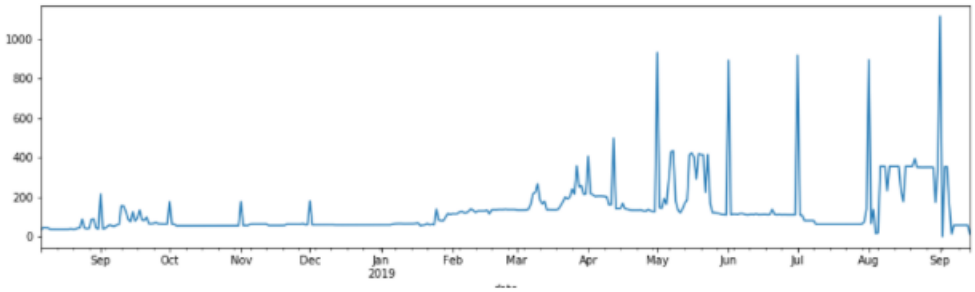


```
plt.show()

# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

# Anomaly Detection
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

Output:



Result:

The above code is executed successfully.